

## Разработка смарт-контрактов

Написание и развертывание смарт-контрактов с использованием Solidity или других языков.

### Инструменты разработки и фреймворки

Для разработки в Ethereum доступен целый ряд инструментов, включая клиентов, интегрированные среды разработки и фреймворки. Мы систематизировали их на рис. 4.

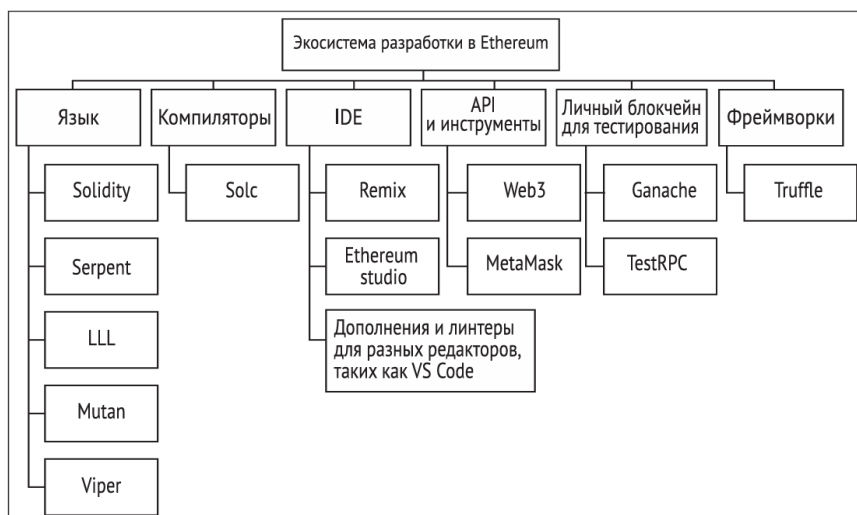


Рис 4 - Разнообразие компонентов для разработки в экосистеме Ethereum

Данная диаграмма охватывает не все фреймворки и инструменты для разработки в Ethereum, а лишь самые популярные из них, а также те, которые мы будем использовать в наших примерах.

### Языки программирования

Смарт-контракты для блокчейна Ethereum можно писать на разных языках программирования. Ниже перечислено пять таких языков:

- **Mutan.** Это язык в стиле Go. Его разработка прекратилась в начале 2015 года, и он больше не используется;
- **LLL.** Расшифровывается как Low-level Lisp-like Language (низкоуровневый лиспоподобный язык). Тоже больше не используется;
- **Serpent.** Это простой и элегантный язык, похожий на Python. Он больше не используется для написания контрактов и не поддерживается сообществом;
- **Solidity.** Этот язык фактически является стандартом для написания контрактов в Ethereum. В этой главе ему уделяется основное внимание. Он будет подробно рассмотрен в дальнейших разделах;
- **Vyper.** Экспериментальный язык, похожий на Python, который должен сделать разработку смарт-контрактов безопасной, простой и хорошо проверяемой.

### Компиляторы

Компиляторы используются для преобразования высокоуровневого исходного кода контракта в формат, совместимый со средой выполнения Ethereum. Компилятор Solidity является самым распространенным, поэтому мы сосредоточимся на нем.

#### Компилятор Solidity (solc)

## Установка в Linux

```
$ sudo apt-get install solc
```

Если репозитории еще не сконфигурированы, это можно легко исправить:

```
$ sudo add-apt-repository ppa:ethereum/ethereum
```

```
$ sudo apt-get update
```

\$ solc --version

## solc, the solidity compiler commandline interface

Version: 0.4.19+commit.c4cbbb05.Darwin.appleclang

В macOS компілятор `solc` можна встановити такими командами:

\$ brew tap ethereum/ethereum

\$ brew install solidity

\$ brew linkapps solidity solc поддерживает множество функций. Некоторые из них продемонстрированы ниже.

- Вывод контракта в двоичном формате:

```
$ solc --bin Addition.sol
```

На рис. 5 показан примерный вывод, который вы получите. Это двоичное представление кода контракта `Addition.sol`.

```
imrans-MacBook-Pro:~ drequinox$ solc --bin Addition.sol  
===== Addition.sol:Addition =====  
  
Binary:  
606060405234156100f5760080fd5b61010b8061001e6000396000f3006060604052600436106049576000357c01000000  
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000  
607d575b600080fd5b3415605857600080fd5b607b600480803560ff1690602001909190803560ff16906020019091905050  
60a9565b005b3415608757600080fd5b608d60c9565b604051808260fff1660ff16815260200191505060405180910390f35b  
808201600080610100a81548160ff021916908360ff1602179055505050565b6000806000905490610100a900460ff1690  
50905600a165627a7a7230582037bbf1721ae442876d01fa64f7fee6baac85d55db40825cf6dea392487369e0029  
imrans-MacBook-Pro:~ drequinox$
```

Рис 5 - Двоичный вывод компилятора Solidity

- Оценка потребления газа:
- `$ solc --gas Addition.sol`
- Эта команда выдает следующий вывод (рис. 6).



программным кодом смарт-контракта. Для взаимодействия с контрактом, развернутым в блокчейне Ethereum, внешним программам необходим его адрес и ABI-интерфейс.

solc является очень мощной командой со множеством параметров, ознакомиться с которыми можно с помощью флага --help. Но в большинстве случаев для разработки и развертывания вам будет достаточно команд, приведенных выше, которые выполняют компиляцию, генерацию ABI-интерфейса и оценивают потребление газа.

## Интегрированные среды разработки

Для разработки на языке Solidity есть разные интегрированные среды разработки (англ. Integrated Development Environment, или IDE). Многие из них доступны в интернете и имеют веб-интерфейсы. Наибольшей популярностью для разработки и отладки смарт-контрактов пользуется среда Remix (ранее из вестная как browser-solidity). Ее мы здесь и обсудим.

### Remix

Remix – это среда для разработки и тестирования контрактов на языке Solidity, основанная на веб-технологиях. Она имеет богатый набор возможностей, которые недоступны в реальном блокчейне; фактически это виртуальная среда, в которой можно развертывать, тестировать и отлаживать свои контракты.

На рис. 8 показано, как она выглядит.

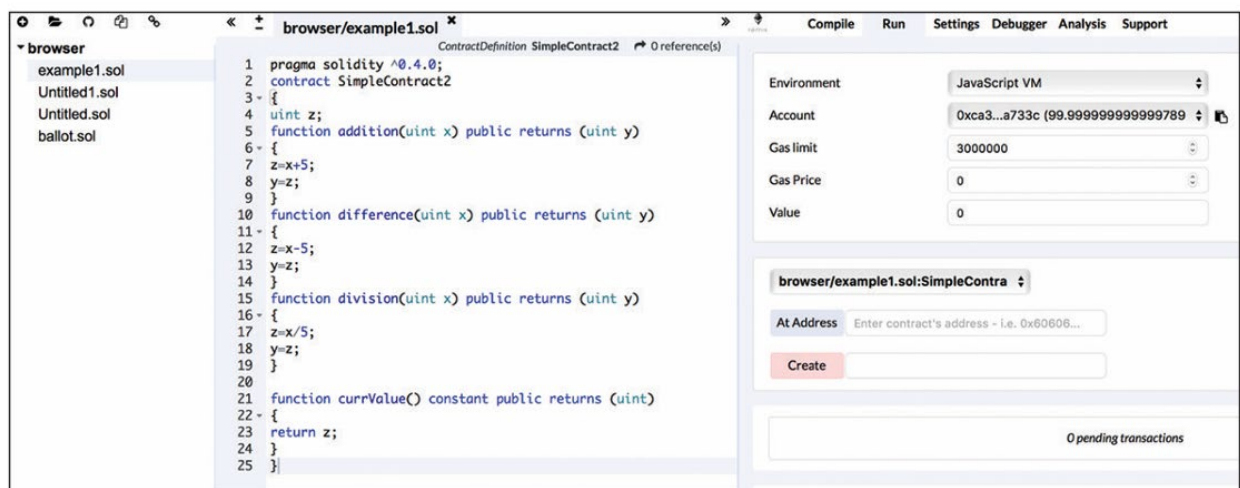


Рис 8 - Remix IDE

Слева находится редактор кода с подсветкой синтаксиса и форматированием, а в правой части доступен ряд инструментов, предназначенных для развертывания, отладки и тестирования контрактов, а также для взаимодействия с ними.

Remix поддерживает дополнительные возможности, такие как взаимодействие с транзакциями, подключение к виртуальной машине JavaScript, настройка среды выполнения, отладчик, формальная проверка и статический анализ. Они совместимы с JavaScript VM (где в качестве среды выполнения выступают Mist, MetaMask или что-то другое) или провайдером Web3, который позволяет подключаться к локальному клиенту Ethereum (такому как geth) через IPC или RPC по HTTP (конечная точка провайдера Web3).

Remix также содержит очень мощный отладчик для EVM, позволяющий выполнять подробные трассировку и анализ байт-кода. Пример показан на рис. 9.

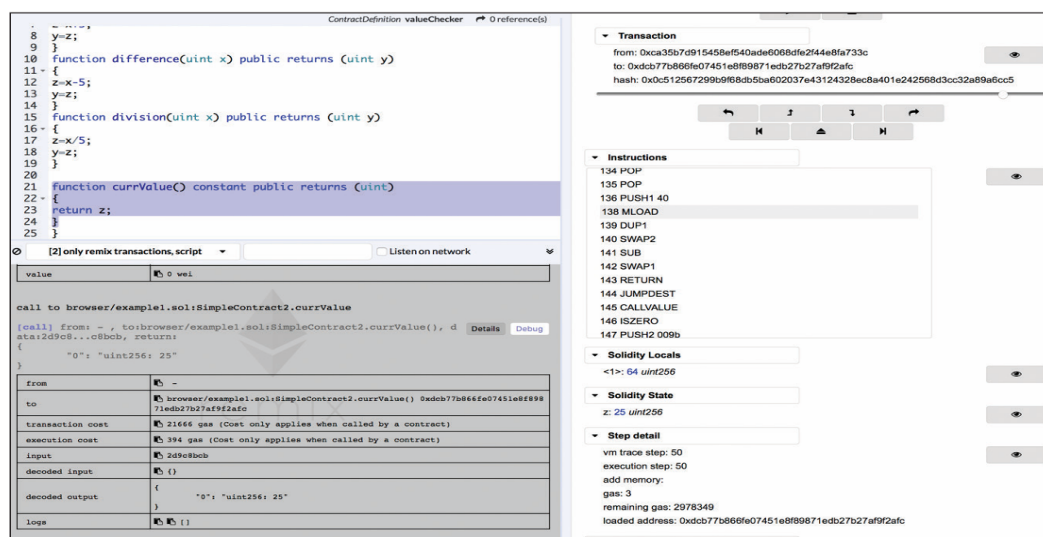


Рис 9 - Отладка в Remix IDE

Здесь мы видим разные элементы Remix IDE. В левой части находится исходный код. Снизу от него размещен журнал с сообщениями и данными, от носящимися к компиляции и выполнению контракта. На снимке экрана, представленном ниже, более подробно показан отладчик Remix. Исходный код переводится в инструкции EVM. Пользователь может по шагово пройти по этим инструкциям и исследовать исходный код, который при этом выполняется (рис. 10).

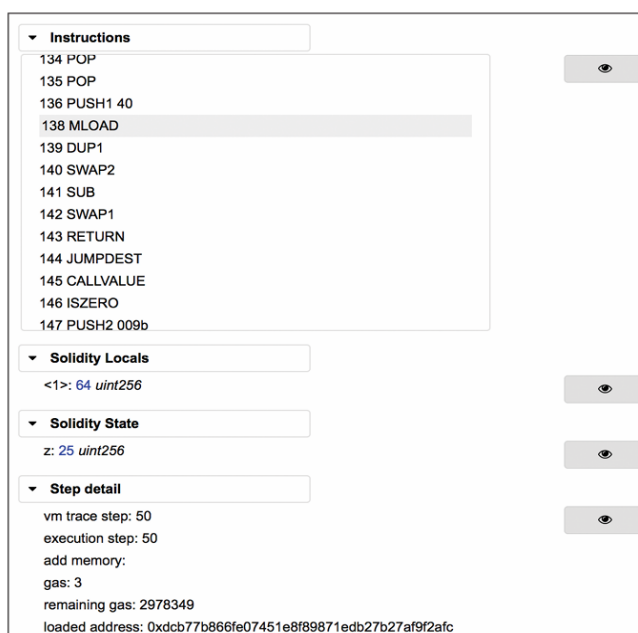


Рис 10 - Отладчик Remix

## Инструменты и библиотеки

Для Ethereum создано множество инструментов и библиотек. В этом разделе мы обсудим самые распространенные из них. Сначала нужно установить программное обеспечение, необходимое для разработки в Ethereum. Во-первых, это Node.js. Node.js версии 7 Пакет Node.js требуется для большинства инструментов и библиотек. Его можно установить с помощью следующих команд: `$ curl -sL https://deb.nodesource.com/setup_7.x | sudo -E bash - sudo apt-get install -y nodejs EthereumJS` Иногда проверка кода в тестовой сети не

представляется возможной, а основная сеть – естественно, не место для тестирования контрактов. Настройка частных сетей может отнимать слишком много времени. В таких ситуациях, когда вам нужно быстро что-то проверить, пригодится эмулятор TestRPC. Он симулирует поведение клиента geth с помощью EthereumJS и ускоряет процесс тестирования. TestRPC можно загрузить через npm в виде пакета для Node.js.

Прежде чем устанавливать TestRPC, следует убедиться в том, что у вас уже установлены Node.js и диспетчер пакетов npm. Установка TestRPC выполняется следующей командой: `$ npm install -g ethereumjs-testrpc` Для запуска testrpc просто введите одноименную команду. Затем откройте другой терминал, а эмулятор пусть работает в фоновом режиме: `$ testrpc` На рис. 13.8 показана информация, которую выводит TestRPC во время работы. Эмулятор автоматически генерирует десять учетных записей и закрытых ключей, а также кошелек HD. Затем он начинает отслеживать входящие подключения на TCP-порту 8545.

```
EthereumJS TestRPC v6.0.3 (ganache-core: 2.0.2)

Available Accounts
=====
(0) 0x6ca19d903eb53e00bb73622d275c965f2abad3d8
(1) 0x1f192daefa61ae050332e6a965e71fcf4621e887
(2) 0x97c0b2ea19a5b496e314e55d1e5a3a5d41b5ad21
(3) 0x3a04fbc6f8eb34b89918628a5a5fde4267e32e28
(4) 0x43e03d85a8a9328f510732be594993ac7011335c
(5) 0x6dfe1a7059df7a625c1ffaed0e97c42384b68446
(6) 0xb9992f167e68dc4bd4a1ce79c07b6193c4e72f37
(7) 0x46243dfcfb6d2d4ec60aa97ebbcac0f96aa33ab
(8) 0xe5b9c05dcb55ad987a504da7fb3dde4281d73bc4
(9) 0x37f6576fd633d95cbc29db28bbae4a272fe5594c

Private Keys
=====
(0) c82c6a860eeb57c8eedbd2e8bc59dc7c800f99118b7f1ef5540c41cdb10805dc
(1) 144271a65d21c59bd6f321659798b42e3d1a22feeb45c2c44f823db0477f330d
(2) d2a55f4406b23c8c18c55a6d30f4b4982ab17a01a6125f0d091e0d4807346905
(3) 1c16608a159b52ba84a0ae170d7642f3166712514693498109640dccc4cae8b9
(4) b7dea27d5bd105bb3e4fbf69598b561563557d343d4c89ea2d7d689f5a160554
(5) 10d467570c50e103ade3694ef85cf9ae0f14cf331ddcd9faaae1f752ed766c5
(6) 7571ecee88840db22a09d8e062292c1e3d106c9e9d8d634f05d4524e75bfa50a
(7) 15e215703ba63d52c870392086f3474b78b5a1b0b6f276fe48c9aae6061f478d
(8) 5dd1fd136b3ba917922b011daaf55ce2b5fd3e332a1f0d39ad5bef664190ebdb
(9) 30e1850a76ee65fcd565caf81fa7310ff239679816be51f0b04149afe4407e1

HD Wallet
=====
Mnemonic:      prepare flavor identify liquid twice tip bullet blanket vast vivid hunt now
Base HD Path:  m/44'/60'/0'/0/{account_index}

Listening on localhost:8545
█
```

Рис 11 - TestRPC

## Ganache

**Ganache** – это новейшее пополнение в арсенале инструментов для разработки и библиотек для Ethereum. Это своего рода замена TestRPC с дружественным пользовательским интерфейсом, который позволяет просматривать подробности о транзакциях и блоках. Это полноценная версия блокчейна Byzantium, которая используется в качестве локальной среды для тестирования. Ganache – это реализация Ethereum на языке JavaScript со встроенным обозревателем и поддержкой майнинга. Она сильно упрощает тестирование в локальных системах. Как видно по рис. 12, клиент Ganache позволяет подробно просматривать транзакции, блоки и адреса.



| ACCOUNTS                                                                                      | BLOCKS                   | TRANSACTIONS         | LOGS               | SEARCH FOR BLOCK NUMBERS OR TX HASHES |                                                |
|-----------------------------------------------------------------------------------------------|--------------------------|----------------------|--------------------|---------------------------------------|------------------------------------------------|
| CURRENT BLOCK<br>0                                                                            | GAS PRICE<br>20000000000 | GAS LIMIT<br>6721975 | NETWORK ID<br>5777 | RPC SERVER<br>HTTP://127.0.0.1:7545   | MINING STATUS<br>AUTOMINING                    |
| <b>MNEMONIC</b><br>candy maple cake sugar pudding cream honey rich smooth crumble sweet treat |                          |                      |                    |                                       | <b>HD PATH</b><br>m/44'/60'/0'/0/account_index |
| ADDRESS<br>0x627306090abaB3A6e1400e9345bC60c78a8BEf57                                         | BALANCE<br>100.00 ETH    | TX COUNT<br>0        | INDEX<br>0         |                                       |                                                |
| ADDRESS<br>0xf17f52151EbeF6C7334FAD080c5704D77216b732                                         | BALANCE<br>100.00 ETH    | TX COUNT<br>0        | INDEX<br>1         |                                       |                                                |
| ADDRESS<br>0xC5fdf4076b8F3A5357c5E395ab970B5B54098Fef                                         | BALANCE<br>100.00 ETH    | TX COUNT<br>0        | INDEX<br>2         |                                       |                                                |
| ADDRESS<br>0x821aEa9a577a9b44299B9c15c88cf3087F3b5544                                         | BALANCE<br>100.00 ETH    | TX COUNT<br>0        | INDEX<br>3         |                                       |                                                |
| ADDRESS<br>0x0d1d4e623D10F9FBA5Db95830F7d3839406C6AF2                                         | BALANCE<br>100.00 ETH    | TX COUNT<br>0        | INDEX<br>4         |                                       |                                                |

Рис 12 - Ganache, персональный блокчейн Ethereum

## MetaMask

MetaMask позволяет взаимодействовать с блокчейном Ethereum из браузеров Firefox и Chrome (рис. 13). Этот инструмент внедряет объект web3 в JavaScript-контекст веб-сайтов, создавая тем самым интерфейс для приложений DApp. Благодаря этому внедрению децентрализованные приложения могут напрямую взаимодействовать с блокчейном.

MetaMask также позволяет управлять учетными записями. Перед тем как попасть в блокчейн, каждая транзакция проходит проверку. Пользователю предоставляется безопасный интерфейс, в котором он может просмотреть транзакцию и решить, следует ее одобрить или отклонить.

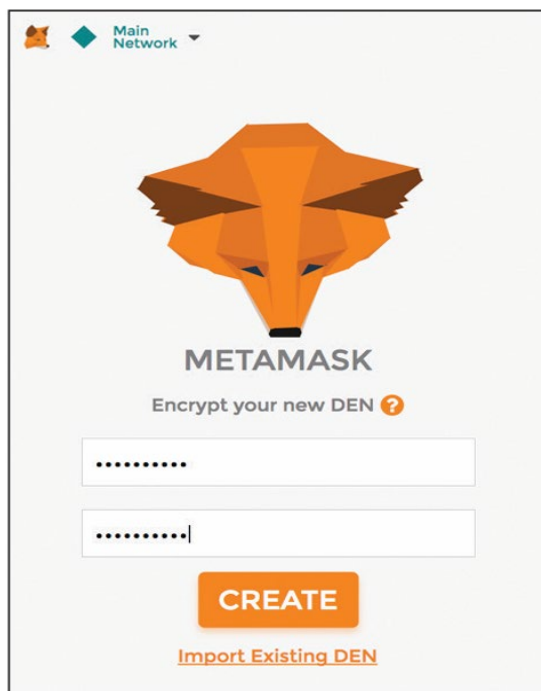


Рис 13 - MetaMask

Как видно по рис. 14, MetaMask умеет подключаться к разным сетям Ethereum. Пользователь сам может выбрать подходящую сеть.

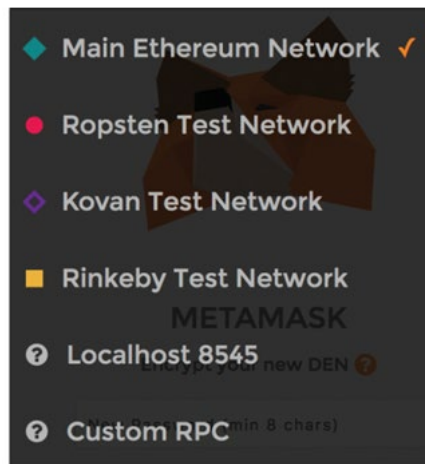


Рис 14 - Выбор сети в пользовательском интерфейсе MetaMask

Клиент MetaMask имеет одну интересную возможность: он способен подключаться к любому нестандартному RPC-интерфейсу прямо из браузера. Это позволяет выбрать для подключения собственный блокчейн, работающий в локальной или удаленной частной сети. С его помощью можно также подключиться к локальному эмулятору блокчейна, такому как Ganache или TestRPC. MetaMask поддерживает управление учетными записями и записывает все их транзакции. Это продемонстрировано на рис. 15.

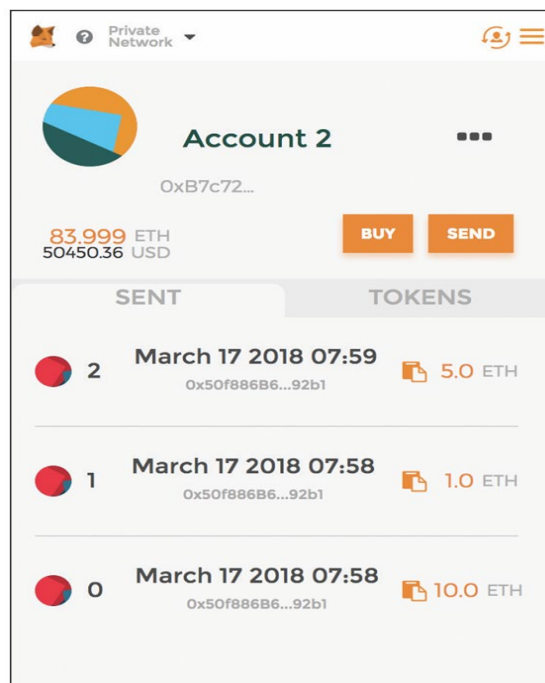


Рис 15 - Просмотр учетных записей и транзакций в MetaMask

## Truffle

Truffle ([truffleframework.com](https://truffleframework.com)) – это среда разработки, которая облегчает и упрощает тестирование и развертывание контрактов в Ethereum. Truffle обеспечивает компиляцию и компоновку кода, а также предоставляет автоматизированный фреймворк для тестирования на основе Mocha и Chai. Он позволяет легко развернуть контракт в любой частной, публичной или тестовой сети Ethereum. Разработчикам предоставляется конвейер, который упрощает обработку всех файлов JavaScript и подготавливает их для использования в браузере.



## Установка

Перед установкой убедитесь в том, что у вас установлен пакет Node.js, так как без него нельзя будет установить truffle:

```
$ node -version
```

```
v7.2.1
```

Для установки truffle достаточно одной команды npm (Node Package Manager – диспетчер пакетов Node.js): `$ sudo npm install -g truffle`

Это займет несколько минут; проверить успешность установки и получить справочную информацию можно будет с помощью команды truffle:

```
$ sudo npm install -g truffle
```

Password:

```
/usr/local/bin/truffle ->
```

```
/usr/local/lib/node_modules/truffle/build/cli.bundled.js
```

```
/usr/local/lib
```

└─ truffle@4.0.1 Введите truffle в терминале, чтобы узнать, как использовать эту команду:  
\$ truffle В ответ будет отображен вывод, показанный на рис. 16.

```
Truffle v4.0.1 - a development framework for Ethereum
Usage: truffle <command> [options]

Commands:
  init      Initialize new Ethereum project with example contracts and tests
  compile   Compile contract source files
  migrate   Run migrations to deploy contracts
  deploy    (alias for migrate)
  build     Execute build pipeline (if configuration present)
  test      Run Mocha and Solidity tests
  debug     Interactively debug any transaction on the blockchain (experimental)
  opcode    Print the compiled opcodes for a given contract
  console   Run a console with contract abstractions and commands available
  develop   Open a console with a local TestRPC
  create    Helper to create new contracts, migrations and tests
  install   Install a package from the Ethereum Package Registry
  publish   Publish a package to the Ethereum Package Registry
  networks  Show addresses for deployed contracts on each network
  watch     Watch filesystem for changes and rebuild the project automatically
  serve     Serve the build directory on localhost and watch for changes
  exec      Execute a JS module within this Truffle environment
  unbox    Unbox Truffle project
  version   Show version number and exit

See more at http://truffleframework.com/docs
```

Рис 16 - Справочная информация True

Как вариант вы можете клонировать репозиторий по адресу [github.com/ConsenSys/truffle](https://github.com/ConsenSys/truffle) и установить truffle вручную. В этом вам поможет утилита Git:

```
$ git clone https://github.com/ConsenSys/truffle.git
```

## Разработка и развертывание контрактов

Разработка и развертывание контракта требуют выполнения различных шагов. В целом этот процесс можно разделить на четыре этапа: написание кода, тестирование, проверка и развертывание. При необходимости в конце можно создать графический интерфейс и сделать его доступным для конечных пользователей с помощью веб-сервера. Обычно веб-интерфейс требуется для взаимодействия с контрактами; он не нужен в контрактах, которые не принимают ввод со стороны пользователей и не производят мониторинг.

## Написание кода

Этот этап посвящен написанию исходного кода контракта на языке Solidity. Это можно сделать в любом текстовом редакторе. Существует множество дополнений для Vim, Atom и других редакторов, которые предоставляют подсветку синтаксиса и форматирование кода для Solidity. Часто для разработки на языке Solidity используют среду Visual Studio Code, которая в последнее время стала довольно популярной. Для этого создано специальное дополнение с поддержкой подсветки синтаксиса, форматирования и интеллектуального анализа. Его можно установить в разделе Extensions (Расширения) прямо в VS Code (рис. 17).

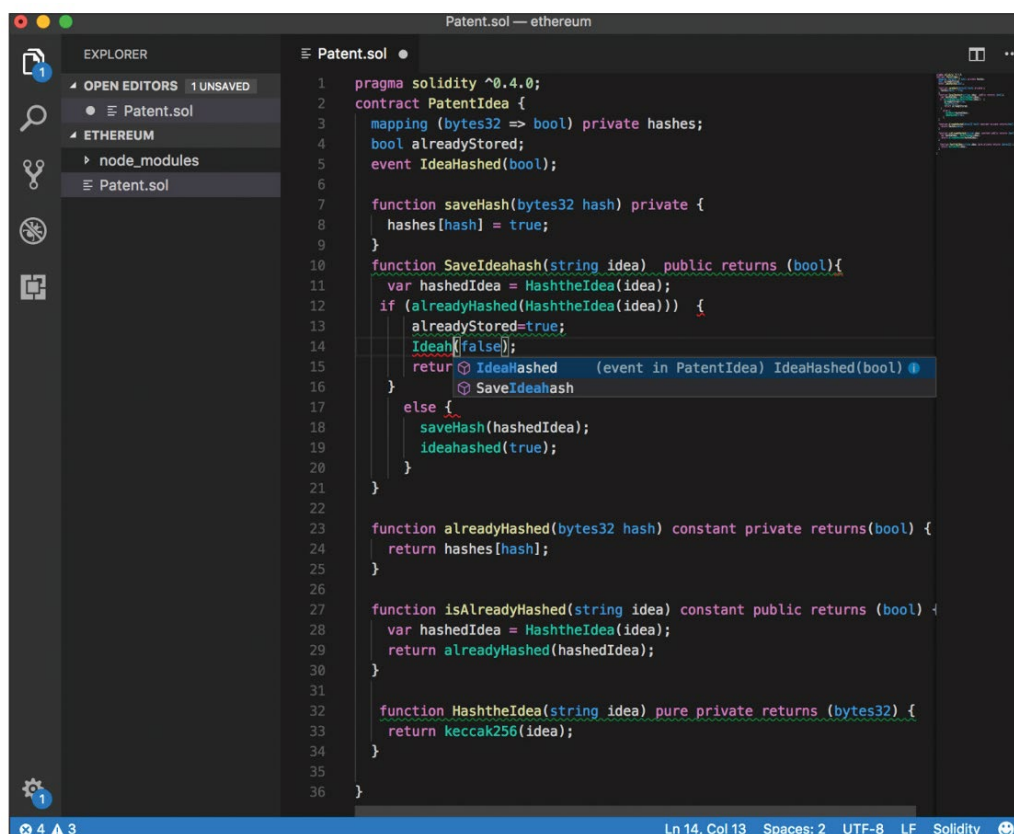


Рис 17 - Visual Studio Code

## Тестирование

Тестирование, как правило, выполняется автоматизированными средствами. Чуть выше вы познакомились с системой Truffle, которая тестирует контракты с помощью фреймворка Mocha. Но вы также можете выполнять ручное функциональное тестирование с использованием Remix, вызывая отдельные функции и проверяя результаты. Мы поговорим об этом в главе 14. После тестирования, проверки корректности и работоспособности в виртуальной среде (такой как TestRPC на основе EthereumJS или Ganache) в рамках частной сети контракт может быть развернут сначала в публичном тестовом блокчейне, а затем в основной сети Ethereum (Byzantium). В следующем разделе вы познакомитесь с языком Solidity и получите базовые навыки для написания контрактов. Это довольно простой язык, который по своему синтаксису похож на C и JavaScript. Все этапы, упомянутые выше, включая проверку, разработку и создание веб интерфейса, будут рассмотрены в следующей главе.

## Язык программирования Solidity

Solidity – это предметно-ориентированный язык, который чаще всего используется для создания контрактов в Ethereum. Несмотря на существование других альтернатив, таких как

Serpent, Mutan и LLL, на момент написания этой книги наибольшей популярностью пользуется именно Solidity. Синтаксис этого языка напоминает JavaScript и C.

На протяжении последних нескольких лет Solidity становится зрелым и простым в использовании языком, но с точки зрения продвинутой, соблюдения стандартов и количества возможностей ему еще далеко до таких устоявшихся языков, как Java, C или C#. Тем не менее это наиболее широко используемое средство для программирования контрактов на сегодняшний день.

Это статически типизированный язык – то есть типы переменных проверяются на этапе компиляции. Каждая переменная, глобальная или локальная, должна иметь явно указанный тип. Благодаря этому все проверки выполняются компилятором, что позволяет выявлять такие ошибки, как интерпретация типов данных на ранних стадиях разработки; делать это на этапе выполнения было бы довольно накладно, особенно учитывая парадигму блокчейна и смарт-контрактов. Среди других особенностей Solidity можно выделить наследование, поддержку библиотек и возможность определения составных типов данных.

Язык Solidity также является контрактно-ориентированным. То есть контракты в нем играют ту же роль, что и концепция классов в объектно-ориентированных языках.

## Типы

Типы данных в Solidity делятся на две категории: примитивные (простые) и ссылочные.

### Примитивные типы

Здесь подробно описываются типы этой категории.

#### Булевы значения

Этот тип данных имеет два возможных значения: true и false. Например: `bool v = true;` `bool v = false;` Эти выражения присваивают переменной `v` значения true и false.

#### Целые числа

Данный тип данных представляет целые числа. В табл. 1 перечислены ключевые слова, которые используются для объявления этих значений.

Таблица 1. Ключевые слова, которые используются для объявления значений

| Ключевое слово    | Тип                     | Подробности                                                                                                                                                                                 |
|-------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>int</code>  | Знаковое целое число    | От <code>int8</code> до <code>int256</code> с шагом 8. Например, <code>int8</code> , <code>int16</code> , <code>int24</code> и т. д.                                                        |
| <code>uint</code> | Беззнаковое целое число | <code>uint8</code> , <code>uint16</code> , ... и до <code>uint256</code> . Беззнаковые целые числа от 8 до 256 бит. Выбор типа зависит от того, сколько бит необходимо хранить в переменной |

В следующем коде, к примеру, `uint` является псевдонимом для `uint256`:

```
uint256 x;
```

```
uint y;
```

```
uint256 z;
```

Данные типы можно объявлять в сочетании с ключевым словом `constant`; в этом случае компилятор не выделит переменной слот для хранения данных, а просто подставит вместо нее реальное значение:

```
uint constant z=10+10;
```

Переменные состояния (или глобальные переменные) объявляются за пределами тела функции и остаются доступными на протяжении всей работы контракта (с учетом типа доступа, который им назначен).

### Адреса

Этот тип данных хранит значение длиной 160 бит (20 байт). Он состоит из нескольких полей, с помощью которых можно запрашивать контракты и взаимодействовать с ними:

баланс. Поле `balance` возвращает баланс адреса в Wei;

функция `send`. Это поле используется для отправки определенного объема Ether по заданному адресу (стандартному для Ethereum 160-битному адресу) и возвращает `true` или `false` в зависимости от результата транзакции. Например:

```
address to = 0x6414cc08d148dce9ebf5a2d0b7c220ed2d3203da; address from = this; if (to.balance < 10 && from.balance > 50) to.send(20);
```

- функции вызова. Для взаимодействия с функциями, у которых нет ABI интерфейса, предусмотрены вызовы `call`, `callcode` и `delegatecall`. Они влияют на безопасность типов и самих контрактов, поэтому использовать их следует с осторожностью; массивы байтов (фиксированной и динамической длины). Solidity поддерживает
- массивы фиксированной и динамической длины. Чтобы указать фиксированную длину, используются ключевые слова `bytes1 ... bytes32`, а для динамических массивов предусмотрены типы `bytes` и `string`. Массивы типа `bytes` хранят необработанные байты, тогда как тип `string` предназначен для хранения строк в кодировке UTF-8. Все эти массивы возвращаются по значению, а обращение к ним приводит к потреблению газа. У всех типов массивов есть поле `length`, которое возвращает их длину.

Пример статического массива (фиксированного размера):

```
bytes32[10] bankAccounts;
```

Пример массива динамического размера:

```
bytes32[] trades;
```

Ниже показан код, который получает длину `trades`:

```
trades.length;
```

### Литералы

Это неизменяемые значения. В следующих подразделах описываются разные виды литералов.

#### Целочисленные литералы

Целочисленный литерал – это последовательность десятичных цифр в диапазоне 0–9. Например: `uint8 x = 2;`

#### Строковые литералы

Строковые литералы определяют последовательность символов, записанных в одинарных или двойных кавычках. Например:

```
'packt' "packt"
```

#### Шестнадцатеричные литералы

Шестнадцатеричные литералы начинаются с ключевого слова `hex` и заключаются в двойные или одинарные кавычки. Например:

```
(hex'AABVCC');
```

## Перечисления

Перечисления дают возможность пользователям определять собственные типы. Например:

```
enum Order {Filled, Placed, Expired }; Order private ord; ord=Order.Filled;
```

В перечислениях позволено явное приведение всех целочисленных типов.

## Функции

Существует два вида функций: внутренние и внешние.

### Внутренние функции

Их можно использовать в контексте текущего контракта.

### Внешние функции

Могут вызываться извне.

В Solidity функция может быть помечена как константа. В этом случае ей будет запрещено вносить любые изменения в контракт; при вызове она лишь возвращает значение и не расходует газ. Это практическая реализация концепции вызова, которую мы обсуждали в предыдущей главе.

Объявление функции имеет следующий синтаксис: `function ( ) {internal|external} [constant] [payable] [returns ( )]`

## Ссылочные типы

Как понятно из названия, эти типы передаются по ссылке. Их еще называют сложными, или составными. Мы поговорим о них в следующих подразделах.

## Массивы

Массив представляет собой набор смежных элементов одного размера и типа, размещенных по определенному адресу в памяти. Здесь используется та же концепция, что и в любом другом языке программирования. У массивов есть два поля под названием `length` и `push`:

```
uint[] OrderIds;
```

## Структуры

Эти конструкции можно использовать для объединения разнородных типов данных в единую логическую группу. С их помощью можно определять новые типы, как показано ниже:

```
pragma solidity ^0.4.0;
```

```
contract TestStruct {
```

```
    struct Trade
```

```
    {
```

```
        uint tradeid;
```

```
        uint quantity;
```

```
uint price;  
string trader;  
}
```

```
// Эту структуру можно инициализировать и использовать следующим образом: Trade tStruct  
= Trade({tradeid:123, quantity:1, price:1, trader:"equinox"}); }
```

### **Местоположение данных**

В Solidity можно указать место, где будет храниться тот или иной сложный тип данных. В зависимости от указанной аннотации или значения по умолчанию это может быть либо постоянное хранилище (storage), либо память (memory); эти ключевые слова можно применять к массивам и структурам. Копирование данных между хранилищем и памятью может оказаться довольно накладным, поэтому явное задание местоположения иногда помогает контролировать расход газа. Существует также особое местоположение, calldata, которое используется для хранения аргументов внешних функций. Внутренние функции по умолчанию хранят свои аргументы в памяти, а все их локальные переменные находятся в постоянном хранилище. В хранилище также обязательно помещаются переменные состояния.

### **Ассоциативные массивы**

Ассоциативные массивы используются для привязывания ключа к значению. Например, в ассоциативном массиве, показанном ниже, все элементы изначально равны нулю: mapping (address => uint) offers; Как видите, при объявлении переменной offers используется ключевое слово mapping. Давайте возьмем более очевидный пример: mapping (string => uint) bids; bids["packt"] = 10; Это, в сущности, словарь или хеш-таблица, где строковые значения привязываются к целочисленным. В ассоциативном массиве bids строка packt привязана к числу 10.

### **Глобальные переменные**

Solidity предоставляет ряд переменных, которые всегда доступны в глобальном пространстве имен. Эти переменные предоставляют информацию о блоках и транзакциях. Таким же образом доступны криптографические функции и значения, связанные с адресами.

Часть из этих функций и переменных показана ниже:

```
keccak256(...) returns (bytes32)
```

Эта функция вычисляет хеш заданного аргумента (Keccak-256).

```
ecrecover(bytes32 hash, uint8 v, bytes32 r, bytes32 s) returns (address)
```

Данная функция возвращает адрес открытого ключа, полученного из цифровой подписи на основе эллиптической кривой. block.number

Это поле содержит номер текущего блока

### **Управляющие конструкции**

В языке Solidity доступны такие управляющие конструкции, как if...else, do, while, for, break, continue и return. Они работают точно так же, как в C или JavaScript.

Ниже показаны некоторые примеры:



- if. Если x равно 0, присваиваем 0 переменной y; в противном случае присваиваем 1 переменной z:  
if (x == 0) y = 0; else z = 1;
- do. Инкрементируем x, пока z больше 1:  
do {  
  x++;  
} (while z>1);
- while. Инкрементируем z, пока x больше 0:  
while(x > 0){ z++;  
}
- for, break и continue. Выполняем какую-то работу, пока x меньше или равно 10. Этот цикл for отработает 10 раз и прервется из-за условия if(z == 5):  
for(uint8 x=0; x<=10; x++)  
{  
  // выполняем какую-то работу z++ if(z == 5) break;  
}  
Он продолжит работать похожим образом, но при выполнении условия цикл опять запустится;
- return. Эта инструкция останавливает выполнение функции и, если это требуется, возвращает значение. Например:  
return 0;  
Этот код останавливает выполнение и возвращает 0.

## События

Solidity позволяет записывать в журнал EVM определенные события. Это бывает довольно полезно, когда внешние интерфейсы необходимо уведомлять о каких-то изменениях в контракте. Эти записи хранятся внутри блокчейна в журналах транзакций. К журналам нельзя обращаться из контрактов; они используются для уведомлений об изменении состояния контракта или возникновении какого-то события (выполнение условия). Ниже показан простой пример, в котором событие valueEvent возвращает true, если параметр, переданный функции Matcher, равен или больше 10:

```
pragma solidity ^0.4.0;

contract valueChecker
{
  uint8 price=10;
  event valueEvent(bool returnValue);

  function Matcher(uint8 x) public returns (bool) { if (x>=price) { valueEvent(true); return true; } }
}
```

## Наследование

Solidity поддерживает наследование. Для получения производной контракта используется ключевое слово is. В следующем примере контракт valueChecker2 наследуется от

valueChecker. Полученная производная имеет доступ ко всем членам родительского контракта, которые не являются приватными:

```
contract valueChecker
{
uint8 price = 20;
event valueEvent(bool returnValue);
function Matcher(uint8 x) public returns (bool)
{
if (x>=price)
{
valueEvent(true);
return true;
} } }
contract valueChecker2 is valueChecker
{
function Matcher2() public view returns (uint)
{
return price+10;
}
}
```

Если в этом примере поменять `uint8 price = 20` на `uint8 private price = 20`, это значение не будет доступным контракту `valueChecker2`. Если член контракта объявлен приватным, ни один другой контракт не сможет к нему обратиться. Remix выведет следующее сообщение об ошибке:

browser/valuechecker.sol:20:8: DeclarationError: Undeclared identifier.

```
return price+10;
```

```
^___^
```

## Библиотеки

Библиотеки единовременно развертываются по определенному адресу, после чего их код можно вызывать с помощью таких команд EVM, как `CALLCODE` или `DELEGATECALL`. Основная идея библиотек состоит в многократном использовании (переиспользовании) кода. С точки зрения вызывающей стороны они ведут себя как контракты. Ниже показан пример объявления библиотеки:

library Addition

```
{
function Add (uint x,uint y) returns (uint z)
```

```
{ return x + y; }
}
```

Эту библиотеку можно вызывать в любом участке контракта, как это показано в следующем примере, но сначала ее нужно импортировать:

```
import "Addition.sol"

function Addtwovalues() returns(uint)
{
return Addition.Add(100,100);
}
```

Библиотеки имеют некоторые ограничения. Например, они не могут содержать переменные состояния, наследовать и быть наследованными. Более того, в отличие от контрактов, они не могут получать Ether.

## Функции

Функции в Solidity представляют собой модули кода, связанные с контрактом. При их объявлении указываются имя, параметры (опционально), модификатор доступа, ключевое слово `constant` (опционально) и, в случае необходимости, тип возвращаемого значения. Пример показан ниже:

```
function orderMatcher (uint x)
private constant returns(bool return value)
```

Здесь для объявления функции используется ключевое слово `function`. В качестве имени указано `orderMatcher`; она принимает необязательный параметр `uint x`; `private` – это модификатор доступа или спецификатор, который контролирует доступ к функции из внешних контрактов. Ключевое слово `constant` говорит о том, что функция ничего не меняет в контракте, а всего лишь извлекает из него значение. Опциональное выражение `returns (bool return value)` описывает возвращаемый тип функции.

- Как объявить функцию. Ниже показан синтаксис объявления функции:  

```
function <имя функции> (<параметры>) <спецификатор видимости>
returns (<возвращаемый тип> <имя переменной> )
{
<тело функции>
}
```
- Сигнатура функции. В Solidity для идентификации функций используют сигнатуры. Это первые четыре байта хеша Кесак-256, полученного из их полной сигнатуры. Сигнатуры автоматически выводятся в Remix IDE, как это показано на рис. 13.15. `f9d55e21` – это первые четыре байта 32-байтного хеша Кесак-256, принадлежащего функции `Matcher`.



Рис 18 - Хеш функции в Remix IDE

В этом примере функция `Matcher` имеет хеш сигнатуры `d99c89cb`. Эта информация может пригодиться при построении интерфейсов.

- Входящие параметры функции. Входящие параметры функции объявляются в формате. Чтобы вам было легче понять, приведем пример, в котором функция `checkValues` принимает параметры `uint x` и `uint y`:

```
contract myContract
{
function checkValues(uint x, uint y)
{
}
}
```

- Исходящие параметры функции. Исходящие параметры функции объявляются в формате. В этом примере показана простая функция, возвращающая значение `uint`:

```
contract myContract
{ function getValue() returns (uint z)
{
z=x+y;
}
}
```

В предыдущем примере функция `getValue` возвращает всего одно значение, но таких значений может быть вплоть до 14, и все они могут иметь разные типы данных. При желании имена неиспользованных возвращаемых значений можно опустить.

- Внутренние вызовы функций. Функции в рамках контекста текущего контракта можно вызывать напрямую. Эти вызовы относятся к функциям, которые существуют внутри того же контракта, и представляют собой простые переходы JUMP на уровне байт-кода EVM.
- Внешние вызовы функций. Для обращения к функциям, которые находятся в другом контракте, используются вызовы сообщений. В этом случае все параметры функции копируются в память. Если обращение к внутренней функции выполняется с помощью ключевого слова `this`, такой вызов тоже считается внешним. Переменная `this` – это указатель на текущий контракт. Ее явно можно преобразовать в адрес, члены которого наследуются контрактом.
- Fallback-функция. Это безымянная функция контракта, у которой нет аргументов и возвращаемых данных. Она выполняется при каждом поступлении Ether. Ее наличие является обязательным, если контракт собирается принимать денежные средства; если ее не реализовать, будет сгенерировано исключение, а переданная сумма вернется обратно. Она также выполняется в ситуациях, когда указанная сигнатура не соответствует ни одной другой функции в контракте. Если контракт должен принимать Ether, fallback-функцию следует объявить с модификатором `payable`, иначе она не сможет принять переданные средства. Функцию, представленную ниже, можно вызвать с помощью метода `address`.

`call()`:

```
function ()
{
throw;
}
```

В этом случае, в связи с условиями, описанными выше, вызов fallback функции приведет к выполнению инструкции `throw`. В результате контракт вернется к состоянию на момент, предшествовавший вызову. Вместо `throw` можно было бы использовать другую

конструкцию; например, мы могли бы записать в журнал событие, чтобы оповестить вызывающее приложение о результате вызова.

- **Функции-модификаторы.** Модификаторы изменяют поведение других функций и могут быть вызваны перед их выполнением. Обычно они используются для проверки каких-то условий или данных перед вызовом функции. Символ `_` (подчеркивание) указывается в модификаторах, вместо которых при вызове подставляется тело функции; проще говоря, он обозначает функцию, которая должна охраняться. Эта концепция аналогична охраняющим функциям в других языках.
- **Конструкторы.** Это необязательная функция, которая вызывается при создании контракта и носит его имя. Конструкторы существуют в единственном экземпляре, и их нельзя вызывать вручную в других участках кода. Это означает, что их нельзя перегружать.
- **Спецификаторы видимости функции (модификаторы доступа).** При объявлении функции можно указать один из четырех спецификаторов доступа:
  - `external`. Такие функции доступны из других контрактов и транзакций. Внутри их можно вызывать только в случае использования ключевого слова `this`;
  - `public`. Функции являются публичными по умолчанию. Их можно вызывать как внутри, так и с использованием сообщений;
  - `internal`. Внутренние функции родителя доступны дочерним контрактам;
  - `private`. Приватные функции доступны только тому контракту, в котором они были объявлены.
- **Модификаторы функций:**
  - `pure`. Запрещает чтение/изменение состояния;
  - `view`. Запрещает любое изменение состояния;
  - `payable`. Позволяет передавать вместе с вызовом плату в виде Ether;
  - `constant`. Запрещает изменение состояния.
- **Другие важные ключевые слова.** Ключевое слово `throw` используется для остановки выполнения. В результате все изменения состояния отменяются. При этом потребляется весь оставшийся газ, и инициатор транзакции не получает никакого возмещения.

## Структура исходного файла Solidity

В следующем разделе мы рассмотрим отдельные элементы исходного файла на языке Solidity.

### Версия

Чтобы исключить проблемы с совместимостью, которые могут возникнуть в будущих выпусках `solc`, можно указать необходимую версию компилятора, используя директиву `pragma`:

```
pragma solidity ^0.5.0
```

Это гарантирует, что данный файл не будет компилироваться с использованием версий `solc`, ниже 0.5.0 и выше или равных 0.6.0.

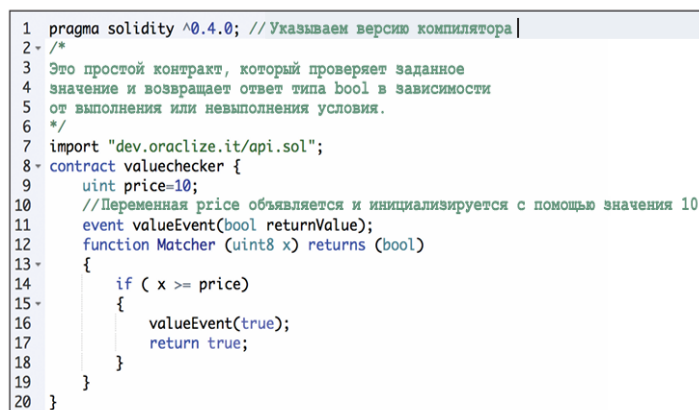
### Импорт

Инструкция `import` позволяет импортировать объекты из существующих файлов Solidity в текущее глобальное пространство имен. Это похоже на одноименную инструкцию в языке JavaScript:

```
import "имя-модуля";
```

### Комментарии

В файлы исходного кода Solidity можно добавлять комментарии, аналогично тому, как это делается в языке C. Многострочные комментарии обрамляются символами `/*` и `*/`, а однострочные начинаются с `//`. На рис. 19 показан пример программы на языке Solidity с использованием `pragma`, `import` и комментариев.



```
1 pragma solidity ^0.4.0; // Указываем версию компилятора |
2 /*
3  Это простой контракт, который проверяет заданное
4  значение и возвращает ответ типа bool в зависимости
5  от выполнения или невыполнения условия.
6  */
7 import "dev.oracize.it/api.sol";
8 contract valuechecker {
9     uint price=10;
10    //Переменная price объявляется и инициализируется с помощью значения 10
11    event valueEvent(bool returnValue);
12    function Matcher (uint8 x) returns (bool)
13    {
14        if ( x >= price)
15        {
16            valueEvent(true);
17            return true;
18        }
19    }
20 }
```

Рис 19 - файлы исходного кода Solidity

### Простая программа на языке Solidity в Remix IDE

На этом мы заканчиваем наше знакомство с Solidity. Это очень богатый язык, который постоянно развивается. Подробная документация и руководства по написанию кода доступны на странице [solidity.readthedocs.io/en/latest/](https://solidity.readthedocs.io/en/latest/).

## Реализация безопасной и эффективной логики смарт-контрактов

### История

Теория смарт-контрактов впервые появилась в 1990-х годах в статье с названием «Формализации и защита связей в публичных сетях» авторства Ника Сабо. Но это было почти за 20 лет до оценки их реального потенциала и пользы с изобретением Bitcoin и последующей разработки в технологии блокчейн. Сабо определяет смарт-контракты следующим образом: «Смарт-контракт – это протокол электронных транзакций, который выполняет условия контракта. Общими целями являются удовлетворение общих условий контракта (таких как условия оплаты, залоговое право, конфиденциальность и даже правоприменение), минимизация возражений, как мошеннических, так и случайных, и минимизация необходимости в доверенных посредниках. Связанные экономические цели включают в себя понижение потерь от мошенничества, расходов на арбитраж и правоприменение и другие затраты на транзакции».

Идея смарт-контрактов была реализована на ограниченном уровне в сети Bitcoin в 2009 году, когда с помощью ограниченного языка скриптов транзакции Bitcoin могут применяться для передачи ценностей между пользователями, через пиринговую сеть, в которой пользователи необязательно доверяют друг другу и нет необходимости в доверенном посреднике.

### Определение

В стандартном определении смарт-контрактов не был достигнут консенсус. Важно определить, чем является смарт-контракт. Ниже вы можете увидеть мою попытку предоставить обобщенное определение смарт-контракта: Смарт-контракт – это безопасная неодолимая компьютерная программа, соответствующая соглашению, которое автоматически выполняется с обеспечением применения силы. Если углубиться в это определение, то можно понять, что смарт-контракт, по сути, является компьютерной программой, написанной на языке, который может быть понят компьютеру или целевому

устройству. Также он выполняет соглашения между сторонами в виде бизнес-логики. Еще одна основополагающая идея состоит в том, что смарт-контракты автоматически выполняются при достижении определенных условий. Они обеспечивают правоприменение, что означает выполнение всех условий контракта в определенной и ожидаемой форме, даже в присутствии оппонента. Правоприменение является более широким понятием, которое обеспечивает традиционное применение законной силы в сочетании с реализацией специфических мер и контроля, призванных для обеспечения возможности выполнения условий контракта без участия какого-либо посредничества. Следует учитывать, что истинный смарт-контракт не должен полагаться на традиционные методы правоприменения. Вместо этого они должны полагаться на принцип «код – это закон», что означает отсутствие необходимости в арбитре или третьей стороне для контроля либо влияния на выполнение смарт-контракта.

Смарт-контракты являются самоприменимыми, в отличие от юридического правоприменения. Такая идея может звучать как мечта либертарианиста, но это абсолютно возможно и соответствует истинному духу смарт-контрактов. Более того, они безопасны и неодолимы, то есть эти компьютерные программы должны быть разработаны в таком виде, чтобы быть отказоустойчивыми и выполнимыми в разумных временных рамках. Эти программы должны работать в здоровом внутреннем состоянии даже при условии неблагоприятных внешних факторов. Например, представьте типичную компьютерную программу, которая имеет определенную логику и выполняется согласно закодированным инструкциям. Однако если среда ее работы или внешние факторы, на которые она опирается, отличаются от нормального или ожидаемого состояния, программа может среагировать своевольно или попросту прекратить свою работу. Важно, чтобы смарт-контракты были устойчивы к такому типу проблем. Безопасные и неодолимые могут считаться требованиями или желаемыми функциями, но это принесет намного больше пользы в долгой перспективе, если свойства безопасности и неодолимости будут включены в определение смарт-контрактов изначально. Это позволит исследователям сфокусироваться на данных аспектах с самого начала и основать солидный базис, от которого смогут оттолкнуться дальнейшие разработки. Также некоторые исследователи предлагают сделать смарт-контракты без автоматического исполнения; вместо этого они предлагают то, что называется автоматизацией, из-за необходимости ручного ввода человеком в некоторых сценариях использования. Например, может потребоваться ручная верификация медицинской записи квалифицированным медицинским профессионалом. В таких случаях цели ком автоматические подходы не приводят к наилучшим результатам. Несмотря на то, что в некоторых случаях ввод и контроль человека желательны, он не является обязательным; и, по мнению автора, чтобы контракт мог называться целиком умным, он должен быть целиком автоматизирован. Случаи необходимости ввода людьми могут и должны быть также автоматизированы с применением Oracles, которые мы обсудим детально далее в этой главе. Смарт-контракты обычно работают путем управления их внутреннего состояния с применением машинной модели состояния. Это позволяет разрабатывать эффективный фреймворк для программирования смарт-контрактов, где состояние контракта продвигается вперед, основываясь на некоторых предустановленных критериях и условиях. Также продолжаются дебаты на тему того, возможно ли принимать код как контракт в суде.

Смарт-контракт отличается от традиционных понятий юриспруденции, хотя они признают и исполняют все формулировки контракта, но суд не понимает код. Эта дилемма поднимает несколько вопросов о том, какую юридическую привязку может иметь смарт-контракт: возможно ли его разрабатывать таким способом, чтобы он был принят и понят в суде? Каким образом реализовать разрешение споров в пределах кода, и возможно ли это в целом? Помимо этого, следует разрешить вопросы требований регулятивных соответствий до того, как смарт-контракты смогут эффективно применяться наравне с традиционными



юридическими документами. Несмотря на то, что смарт-контракты называются умными, фактически они могут делать лишь то, на что запрограммированы, и это является положительной стороной, так как это свойство смарт-контрактов гарантирует им одинаковый вывод при каждом применении. Эта детерминированная природа смарт-контрактов крайне желательна в блокчейн-платформах из-за согласующихся требований консенсуса. Это означает, что смарт-контракты не являются на самом деле умными, они просто делают то, на что запрограммированы. Теперь возникает проблема большого разрыва между реальным миром и миром блокчейна. В этой ситуации натуральные языки не понимаются смарт-контрактом, и аналогичным образом код не понятен в реальном мире. Таким образом, возникает несколько вопросов: каким образом могут работать обычные контракты реального мира в блокчейне? Как построить этот мост между реальным миром и миром смарт-контрактов? Предыдущие вопросы раскрывают различные возможности, такие как создание смарт-контрактов таким образом, чтобы они были читабельны не только устройствами, но и людьми. Если и люди, и устройства смогут понимать код, записанный в смарт-контракте, он сможет получить большее распространение в юридических ситуациях, по сравнению с просто фрагментом кода, который не понимает никто, кроме программистов. Это желаемое свойство является созревшей областью для исследований, и уже было проведено достаточно работ для получения ответов на такие вопросы, как семантика, смысл и интерпретация контракта.

Уже были проведены работы по формальной интерпретации контрактов натурального языка путем комбинирования его с кодом смарт-контрактов через связи условий контракта с элементами, понятными на машинном уровне. Для этого используется язык разметки. В качестве примера такого языка можно привести формат юридического обмена (Legal Knowledge Interchange Format LKIF), который является XML-схемой для представления теорий и доказательств. Он был разработан в рамках проекта ESTRELLA в 2008 году.

Смарт-контракты по своей природе должны быть детерминированными. Это свойство позволит смарт-контрактам работать на любом узле сети и получать одинаковый результат. Если результат будет хоть немного отличаться на разных узлах, консенсус не будет достигнут, что может привести к краху всей парадигмы распределенного консенсуса в блокчейне. Помимо этого, желательно, чтобы язык контракта был сам по себе детерминированным, тем самым гарантируя целостность и стабильность смарт-контрактов. Детерминированность означает отсутствие каких-либо недетерминированных функций, которые могут привести к различным результатам на разных узлах. В качестве примера разные операции с плавающей запятой, вычисляемые разными функциями на разных языках программирования, могут получить разные результаты при работе в разных средах выполнения. Еще один пример: некоторые математические функции в JavaScript могут получать разные результаты для одного ввода в разных браузерах, что может привести к различным неполадкам. Подобные вещи абсолютно нежелательны в смарт-контрактах, так как если результаты не будут совпадать по узлам, они никогда не придут к консенсусу. Свойство детерминированности гарантирует, что смарт-контракты всегда будут производить один и тот же вывод для определенного ввода. Иными словами, программы при выполнении производят надежную и точную бизнес-логику, которая полностью соответствует требованиям, запрограммированным в высокоуровневом коде. Резюмируя, смарт-контракт имеет следующие четыре свойства: автоматическое выполнение; правоприменение; семантически крепкий; безопасный и неодолимый.

Первые два свойства являются минимальными требованиями, когда последние два могут необязательно требоваться или реализоваться в некоторых сценариях и могут быть опущены. Например, контракт финансовых производных необязательно должен быть семантически крепким и завершенным, но должен хотя бы быть автоматически выполняемым и правоприменяемым на фундаментальном уровне. С другой стороны, запись

названия должна быть семантически крепкой и полной, для того чтобы обеспечить понимание и компьютерами, и людьми. Ян Григг решил эту проблему интерпретации в своем изобретении рикардских контрактов, которые мы рассмотрим детально в следующем разделе.

Шаблоны смарт-контрактов. Смарт-контракты могут быть внедрены в любой индустрии, где они необходимы, но большинство сфер применения относится к финансовой отрасли. Это обусловлено тем фактом, что в финансовой индустрии нашлось много применений для технологии блокчейн. Именно в этой отрасли возник высокий интерес к исследованиям задолго до других сфер. Недавние специфические к этой отрасли работы в пространстве смарт-контрактов предложили идею шаблонов смарт-контрактов. Идея состоит в том, чтобы построить стандартные шаблоны для обеспечения фреймворка и поддержки юридических соглашений для финансовых инструментов. Изначально идея была предложена компанией Clack et al. в их материале, опубликованном в 2016 году под названием «Smart Contract Templates: Foundations, design landscape and research directions».

В материале также предлагались доменные языки для разработки и реализации шаблонов смарт-контрактов. Одним из предложенных языков для начала разработки был CLACK, общий язык для знаний расширенных контрактов. Этот язык предназначен для предоставления широкого спектра функций в диапазоне от использования традиционных юридических документов до возможности выполнения на различных платформах и криптографических функций. Последнее время компания Clack et al. проводит работы по разработке шаблонов смарт-контрактов с возможностями юридического правоприменения. Эти предложения обсуждались в их исследовательском материале «Smart Contract Templates: essential requirements and design options».

Основная цель этого материала состоит в исследовании связки юридических документов и кода с помощью языка разметки. Рассматриваются способы создания умных юридических соглашений, а также их форматирования, выполнения и запуска в серийное хранение и передачу. Эти работы продолжаются и сейчас, а данная сфера является открытой для дальнейших исследований и разработки. В финансовой сфере контракты не являются новой концепцией, уже используются DSL различных доменных языков для предоставления определенного языка для определенного домена. Например, существуют DSL для поддержки разработки продуктов страхования, представления производных энергии или применения в постройке стратегий торгов. Также важно понимать концепцию доменных языков, поскольку такой тип языков может использоваться для программирования смарт-контрактов. Эти языки разрабатываются с ограниченными выразительными возможностями для конкретного применения в необходимой сфере. Доменные языки (Domain-specific languages – DSL) отличаются от универсальных языков программирования (General-purpose programming languages – GPL). DSL имеют небольшой набор функций, которых достаточно и которые оптимизированы для того домена, с которым они будут работать, и, в отличие от GPL, обычно не используются для разработки крупных приложений общего назначения. Основываясь на философии устройства DSL, можно предположить, что такие языки будут разрабатываться сугубо для написания смарт-контрактов. Уже выполнена определенная часть работ, и одним из таких языков является Solidity, который был представлен с блокчейном Ethereum для написания смарт контрактов. Еще одним таким языком является Vyper, который был недавно представлен для разработки смарт-контрактов Ethereum. Данная идея языков, основывающихся на доменах для программирования смарт-контрактов, может быть расширена в будущем в графический доменный язык – платформу моделирования смарт-контрактов, в которой доменный эксперт (не программист, например, а торговый представитель) может использовать графический интерфейс и полотно для определения и черчения семантики и производительности финансового контракта. Как только поток был начерчен и завершен, он может быть сначала сэмплирован для проверки и после этого отправлен из той же системы на целевую платформу, которая может быть

блокчейном. Это тоже не является новой концепцией, и похожий подход используется в продукте Tibco StreamBase, который является системой, основанной на Java. Она используется для постройки основанных на событиях высокочастотных торговых систем. Предлагается продолжать исследования в сфере разработки высокоуровневых DSL, которые могут быть применены для программирования смарт контрактов в удобном графическом пользовательском интерфейсе, тем самым позволяя доменному эксперту (не программисту, а юристу) разрабатывать смарт-контракты.

## **Оракулы**

Оракулы являются важным компонентом экосистемы смарт-контрактов. Ограничение смарт-контрактов состоит в том, что они не могут получать доступ к внешним данным, которые могут быть необходимы для контролирования выполнения бизнес-логики, таким как, например, цена акций продукта безопасности, которая требуется в контракте для освобождения выплат дивидендов. Оракулы могут использоваться для предоставления смарт-контрактам внешних данных. Оракул – это интерфейс, который доставляет данные из внешнего источника в смарт-контракт.

В зависимости от отрасли и требований оракулы могут доставлять различные типы данных, от отчетов погоды, новостей реального мира и корпоративных данных до информации с устройств интернета вещей (Internet of Things – IOT). Оракулы являются доверенной стороной, которая использует защищенный канал для передачи данных в смарт-контракт. Оракулы также имеют возможность цифровой подписи данных, которая гарантирует подлинность источника данных. Тогда смарт-контракты могут подписываться на оракулов, запрашивать из них данные, или же сами оракулы могут направлять данные в смарт-контракты. Важным является отсутствие возможности у оракула манипулировать предоставляемыми им данными и оперировать исключительно подлинной информацией. Несмотря на то, что оракулы являются доверенной стороной, в некоторых случаях все равно возможны неточности данных ввиду манипуляций. По этой причине необходимо сделать так, чтобы оракулы не имели возможности изменять данные. Подтверждение данных может быть осуществлено с помощью нотариальных схем, которые будут рассмотрены далее в этой главе. В этом подходе уже наблюдается проблема, которая в некоторых случаях является нежелательной, и это проблема доверия. Насколько вы доверяете третьей стороне в плане качества и подлинности данных, которые она предоставляет?

Это особо важно в финансовом мире, где рыночные данные должны быть точными и надежными. Для разработчика смарт-контракта допустимо принимать данные из оракула, которые предоставлены крупной, уважаемой и доверенной третьей стороной, но проблема централизации остается и в таком случае. Подобные типы оракулов можно называть стандартными, или простыми, оракулами. Например, источником данных может служить уважаемое синоптическое агентство или система информирования аэропортов об отсрочке рейсов. Еще одна концепция для обеспечения надежности данных от третьих сторон для оракулов состоит в том, чтобы получать данные из нескольких источников; даже пользователи или представители общественности, имеющие доступ и представление о некоторых данных, могут предоставлять эти данные. Информация может подлежать агрегации, и если из многих источников получен высокий процент одинаковых данных, это служит гарантией того, что данные точны и им можно доверять. Еще одним типом оракула, который прямым образом произошел от требований децентрализации, является децентрализованный оракул. Эти оракулы могут основываться на каком-нибудь распределенном механизме. Также предусматривается возможность для оракулов находить данные в другом блокчейне, который основывается на распределенном консенсусе, тем самым обеспечивая подлинность данных. Например, структура с запущенными частными блокчейнами может публиковать свои данные через оракула, которые позже могут потребляться другими блокчейнами.

Еще одна концепция аппаратных оракулов была представлена исследователями, для которой требуются данные о реальном мире от физических устройств. Например, это может применяться в телеметрии или интернете вещей. Однако этот подход требует механизма, согласно которому аппаратные устройства являются взломоустойчивыми. Для этого необходимо предоставлять криптографическое доказательство (безотказности и целостности) данных и механизма взломостойкости устройства интернета вещей, который определит устройство бесполезным в случае попытки взлома. В данный момент доступны платформы, обеспечивающие смарт-контрактам получение внешних данных с помощью оракула. Существуют разные методы записи данных оракулом в блокчейн в зависимости от используемого блокчейна. Например, в блокчейне Bitcoin оракул может записывать данные в определенную транзакцию, а смарт-контракт может мониторить эту транзакцию в блокчейне для чтения данных.

Услуги оракула предоставляют различные онлайн-сервисы, такие как [www.oraclize.it](http://www.oraclize.it) и [www.realitykeys.com](http://www.realitykeys.com). Еще один из таких сервисов – [smartcontract.com](http://smartcontract.com) – предоставляет внешние данные и возможность осуществления платежей с помощью смарт-контрактов.

Все эти сервисы имеют цель обеспечить смарт-контракты данными, необходимыми для принятия и выполнения решений. Для доказательства подлинности данных, полученных оракулами из внешних источников, могут применяться такие механизмы, как TLSnotary, которые обеспечивают доказательство факта коммуникации между источником данных и оракулом. Это подтверждает факт того, что данные, отправленные смарт-контракту, получены из источника.

### **Запуск смарт-контрактов в блокчейне**

Смарт-контракты следует или не следует запускать в блокчейне, но это имеет смысл делать ввиду распределенного и децентрализованного механизма консенсуса, предоставляемого блокчейном. Примером платформы блокчейна, которая изначально поддерживает разработку и запуск смарт-контрактов, является Ethereum. В этой сети смарт-контракты обычно являются частью более обширного приложения, такого как децентрализованная автономная организация (ДАО, Decentralized Autonomous organization – DAO). Для сравнения, в блокчейне Bitcoin таймлоки транзакции, такие как поле nLocktime и оператор скрипта CHECKLOCKTIMEVERIFY (CLTV), CHECKSEQUENCEVERIFY, могут быть включателем простой версии смарт-контракта. Эти таймлоки позволяют зафиксировать транзакцию на определенное время или до определенного количества блоков, тем самым запуская простой контракт по следующему механизму: определенная транзакция может быть разблокирована только при достижении определенных условий (время или количество блоков). Например, вы можете реализовать следующие условия: «Заплатить стороне X N биткойнов через 3 месяца». Однако этот подход крайне ограничен и может рассматриваться только как пример базового смарт-контракта. В дополнение к предыдущему примеру для постройки базовых смарт-контрактов может применяться язык скриптов Bitcoin, хоть и в ограниченном виде. В качестве примера базового смарт-контракта: оплатить на тот адрес Bitcoin, который сможет продемонстрировать «коллизионную атаку хеша». Таким был конкурс на форуме Bitcointalk, в котором награду в виде биткойнов мог получить любой, кто смог бы найти коллизии для хеш-функций. Такое условное разблокирование биткойнов только при демонстрации успешной атаки является базовым типом смарт-контракта. Смарт-контракты поддерживаются многими другими платформами блокчейна, такими как Monax, Lisk, Counterparty, Stellar, Hyperledger fabric, corda и Axoni core. Смарт-контракты могут разрабатываться на разных языках. Критическим требованием является детерминированность, так как получение единого результата независимо от места выполнения кода смарт-контракта является очень важным моментом. Это же требование также подразумевает, что код смарт-контракта не имеет ошибок. Для постройки смарт-контрактов были разработаны разные языки, такие как Solidity, работающий в системе

Ethereum Virtual Machine (EVM). Стоит отметить, что уже существуют платформы с поддержкой распространенных языков для разработки смарт-контрактов, такие как Lisk, поддерживающий JavaScript. Еще одним выдающимся примером может служить Hyperledger fabric, который поддерживает Golang, Java и JavaScript для разработки смарт-контрактов.

DAO является одним из масштабнейших проектов краудфандинга, он был запущен в апреле 2016 года. Это был набор смарт-контрактов, написанных для предоставления платформе инвестирования. Из-за ошибки в коде он был взломан в июне 2016 года, и эквивалент 50 миллионов долларов был украден из DAO в другой аккаунт. Несмотря на применение термина взлома, фактически он не был взломан. Смарт-контракт попросту сделал то, что его попросили. Это было непреднамеренное поведение, которое не предусмотрели программисты данного проекта. Данный инцидент привел к жесткой вилке в сети Ethereum для восстановления от атаки. Следует отметить, что понятие код – это закон, или неодолимые смарт-контракты следует рассматривать с долей скептицизма, так как реализация этих концепций еще недостаточно зрелая, для того чтобы заслужить полное и неоспоримое доверие. Свидетельством тому могут служить недавние события, когда организация Ethereum смогла остановить и изменить выполнение проекта DAO с помощью жесткой вилки. Несмотря на то, что эта вилка произошла по истинным причинам, это противоречит духу децентрализации и понятию код – это закон. С другой стороны, сопротивление этой жесткой вилке и решение некоторых майнеров продолжать майнинг исходной цепи привели к созданию валюты Ethereum Classic. Эта цепь является оригинальной цепью Ethereum до вилки, и в ней до сих пор код – это закон. Данная атака указывает на опасность невыполнения полного и формального тестирования смарт-контрактов. Также она указывает на необходимость разработки формального языка для разработки и верификации смарт-контрактов. Таким образом, мы убеждаемся в необходимости тщательного тестирования для избегания проблем, которые произошли с DAO. В сети Ethereum недавно были найдены различные уязвимости вокруг языка разработки смарт-контрактов. По этой причине крайне важно разработать стандартный фреймворк для решения этих проблем. Уже начаты некоторые работы, например онлайн-сервис [securify.ch](https://securify.ch) предоставляет инструменты для формальной верификации смарт-контрактов. Однако эта сфера уже готова к новым исследованиям касательно ограничений языков смарт-контрактов.